COSC204
Computer Systems

Advanced 6809 Assembly

Overview

- Last Lecture
  - Assembly language programming
- This Lecture
  - 6809 addressing modes
  - Calling conventions
  - Local variables
  - Source
    - Lance A. Leventhal (1981) 6809 Assembly Language Programming, Osborne McGraw-Hill, ISBN: 9780079310354
    - Pierre Sarrazin (2022) The CMOC C-like 6809-targeting cross-compiler
- Next Lecture
  - C

Addressing Modes

- Recall that assembly instructions can have operands
  - Those operands are a syntax all their own
  - They describe how the instruction will do its work
  - The different ways they work are called ***addressing modes***

- We've already seen some addressing modes
  - Implied mode

```
DECA      ; subtract 1 from A
```

  - Immediate mode

```
LDA #$45  ; load A with the value $45
```

  - Extended mode

```
LDA $3B45 ; load A with the value at memory location $3B45
```

6809 Addressing Modes

- Motorola list 10 addressing modes
  - Implied
  - Immediate
  - Extended
  - Extended Indirect
  - Direct
  - Register
  - Indexed
    - Zero-offset
    - Constant offset
    - Accumulator offset
    - Auto increment / decrement
  - Indexed indirect
  - Relative
  - Program counter relative

Implied

- These instructions don't take operands
- Examples include:
  - `DECA` ; A = A - 1

Register

- The value is stored in the register specified
- Examples include:
  - `TFR X,Y` ; Y = X

Immediate

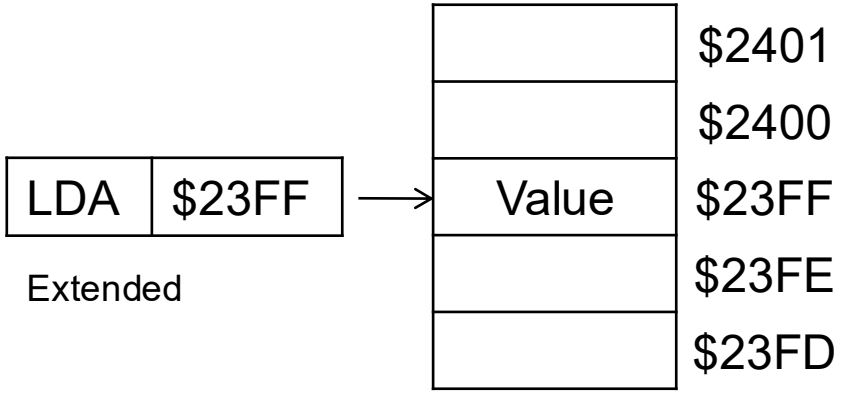
- The value to use is specified as the operand
  - Useful for loading constant values

- Examples include:
  - `LDA #$FF` ; A = 255;

Extended

- The value is stored at the memory location specified in the operand

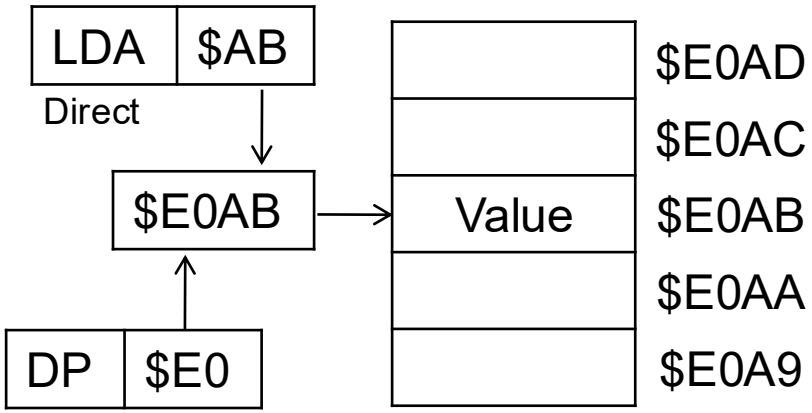
- Examples include:
  - `LDA $23FF` ; A = value of memory location $23FF
  - `LDA HERE`



Direct

- The value is stored at the memory location specified in the operand, on the page specified by `DP`

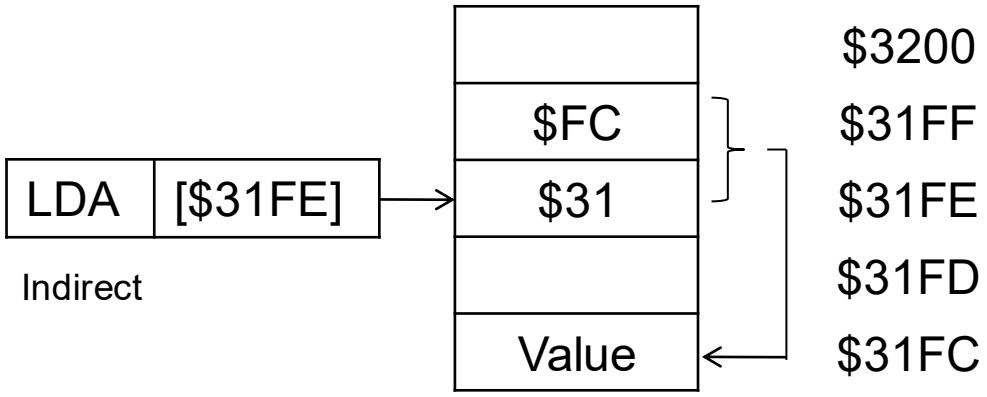
- Examples include:
  - `LDA $FF` ; A = value of memory location (DP)$FF
  - `LDA <HERE` ; This syntax is rare



Extended Indirect

- The value is stored at the memory location specified at the memory location in the operand

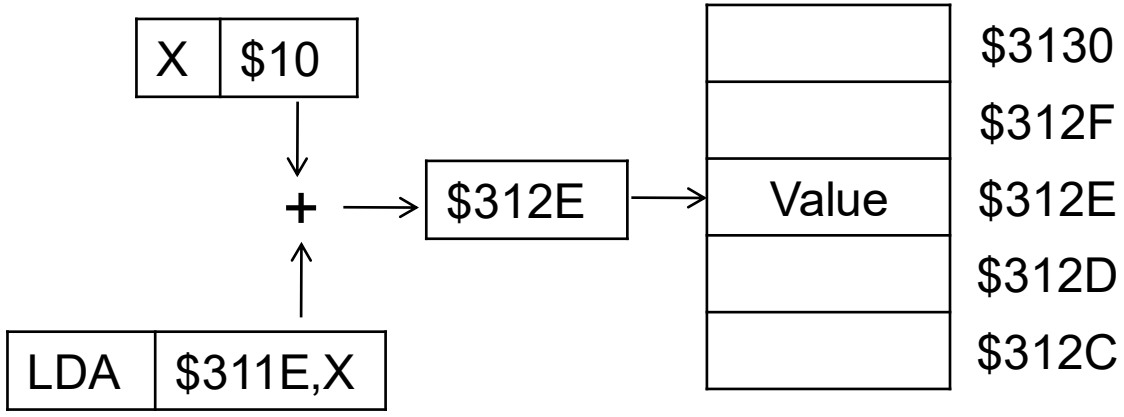
- Examples include:
  - `LDA [$31FE]` ; $31FE says where to get the value from
  - `LDA [HERE]` ; HERE says where to get the value from



Indexed

- The value to use is stored at the memory location that is the sum of the operands

- Examples include:
  - `LDA $311E,X` ; Use memory location ($311E+X)



Indexed

Indexed

- Zero-offset

```
LDA ,X          ; from where X points
```

- Constant offset

```
LDA $23,X       ; from $23 bytes past where X points
```

- Accumulator offset

```
LDA B,X         ; from B bytes past where X points
```

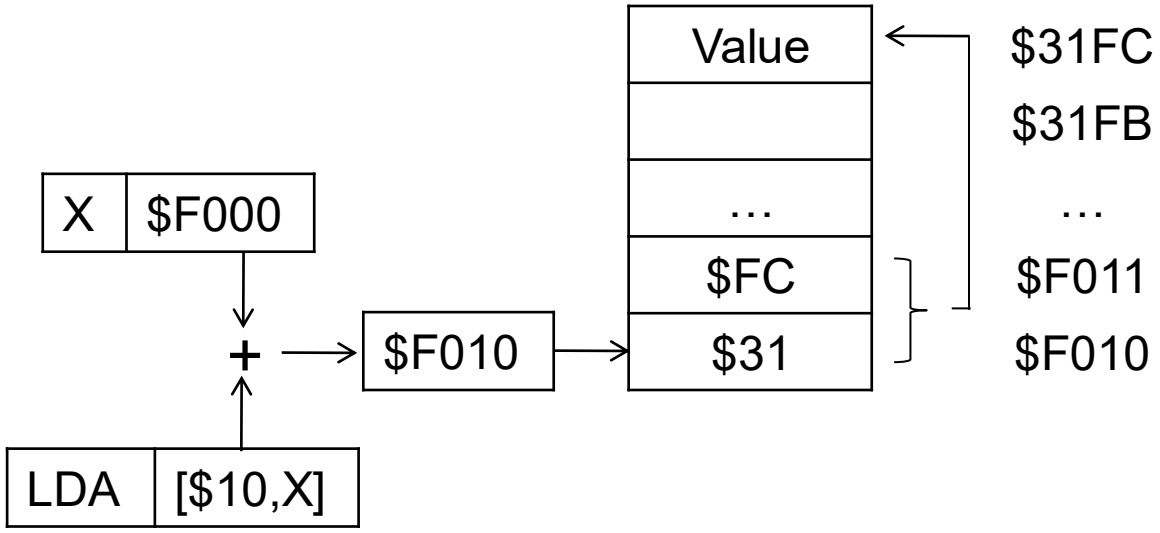
- Auto increment / decrement

```
LDA ,X+         ; from where X points then X=X+1
LDA ,X++        ; from where X points then X=X+2
LDA ,-X         ; X=X-1, then from where X points
LDA ,--X        ; X=X-2, then from where X points
```

Indexed Indirect

- The value is stored at the memory location specified at the memory location in the operand

- `LDA [$10,X]` ; Load A with where $10 past X says



Indexed Indirect

Relative

- The value is relative to here
  - For example, branch backwards by 9 bytes
  - You don't normally think of branches as relative, but they are

- Examples include:
  - `BEQ HERE` ; Branch to HERE if the Z flag is set

- `BEQ` does a short branch (-128 to 127 byte offset)
- `LBEQ` does a long branch (full 16-bit offset)

Program Counter Relative

- The value is stored at the memory location specified in the operand, but computed relative to the program counter

- Examples include:
  - `LDA TABLE,PCR` ; A = the value stored at TABLE

- If *all* memory references are *relative* then the program can be loaded anywhere in memory, and will still work. It is said to be ***relocatable***

6809 Operand Cheat Sheet

Symbol	Meaning
#$Value	Use this value
Label	Get it from (or go to) Label
#Label	The place in memory where Label is (address of Label)
r,r	Use the registers denoted by r
<Label	Use DP as the high byte
[ ]	This address says where to get it from (dereference)
,	Add, e.g. add $10 to X: $10,X
+ or ++	Post increment by 1 or 2
- or --	Pre decrement by 1 or 2

LEA – Load Effective Address

- Load `Y` with the value $1234

```
LDY #$1234
```

- Load `Y` with the value stored at memory[$1234]

```
LDY $1234
```

- Load `Y` with the value stored at memory[`X`+$12]

```
LDY $12,X
```

- Work out where you would load the value from

```
LEAY $12,X
```

- That is, compute `X` + $12 and store that value in `Y`
- **What does `LEAX B,X` do?**

Calling Convention

- Recall, in assembly
  - All registers are global
  - There are no parameters to routines
  - There are no local variables
- Recall, in high level languages
  - Discouraged use of global variables
  - Parameters to routines
  - Local variables
- How is this done?

Calling Convention

- When we wrote “hello world” we used several registers. To save them we pushed them on entry to our routine and pulled on exit

```
PUTS:
    PSHS A,X,Y      ; save A, X, and Y
     . . .
    PULS A,X,Y      ; restore A, X, Y
    RTS             ; return
```

- But what does a compiler do?

Calling Convention (ABI)

- The *calling convention* is the set of rules that describe
  - How to pass parameters
  - How to return a result
  - Which registers a routine may alter
- Let's not reinvent the wheel (as we already did)
- We'll use CMOC, a 6809 C compiler, as an example
  - But it differs from compiler to compiler and OS to OS

CMOC Calling Convention

- A routine must preserve `Y`, `U`, `S` and `DP`
- A routine may change `A`, `B`, `X` and `CC`

- Parameters are pushed on the stack in the reverse order
- The ***caller*** pulls them off the stack after the call returns

- `char` parameters are promoted to `int`
- `unsigned char` are promoted to `unsigned int`

- Return 8-bit values in `B`
- Return 16-bit values in `D`

- We're not going to talk about passing a `struct` by value

Example

- Lets examine the assembly we get for this C

```
uint16_t two_params(uint8_t first, uint16_t second)
{
return first + second;
}

uint16_t call_one(void)
{
return two_params(204, 431);
}
```

- What do we expect?

Calling A Routine

- For the C

```
two_params(204, 431);
```

- Parameters are pushed on the stack in the reverse order, `char` becomes `int`

```
LDD #$01AF          ;decimal 431
PSHS B,A            ;argument 2, int
CLRA
LDB #$CC            ;decimal 204
PSHS B,A            ;argument 1, int
LBSR _two_params
```

- The caller pops them off the stack after the call:

```
LEAS 4,S
```

Preserving Registers

- A routine must preserve `Y`, `U`, `S` and `DP`
  - CMOC ignores `DP`, and rarely uses `Y`
- A routine may change `A`, `B`, `X` and `CC`

```
_two_params
    PSHS U
    LEAU ,S
    …
    LEAS ,U
    PULS U,PC
```

- This Pre-amble saves `U` then copies `S` to `U`
- This post-amble restores `S` then restores `U` (and `PC`)
  - Nice trick, no `RTS` needed!

Accessing Parameters

- For the C

```
return first + second;
```

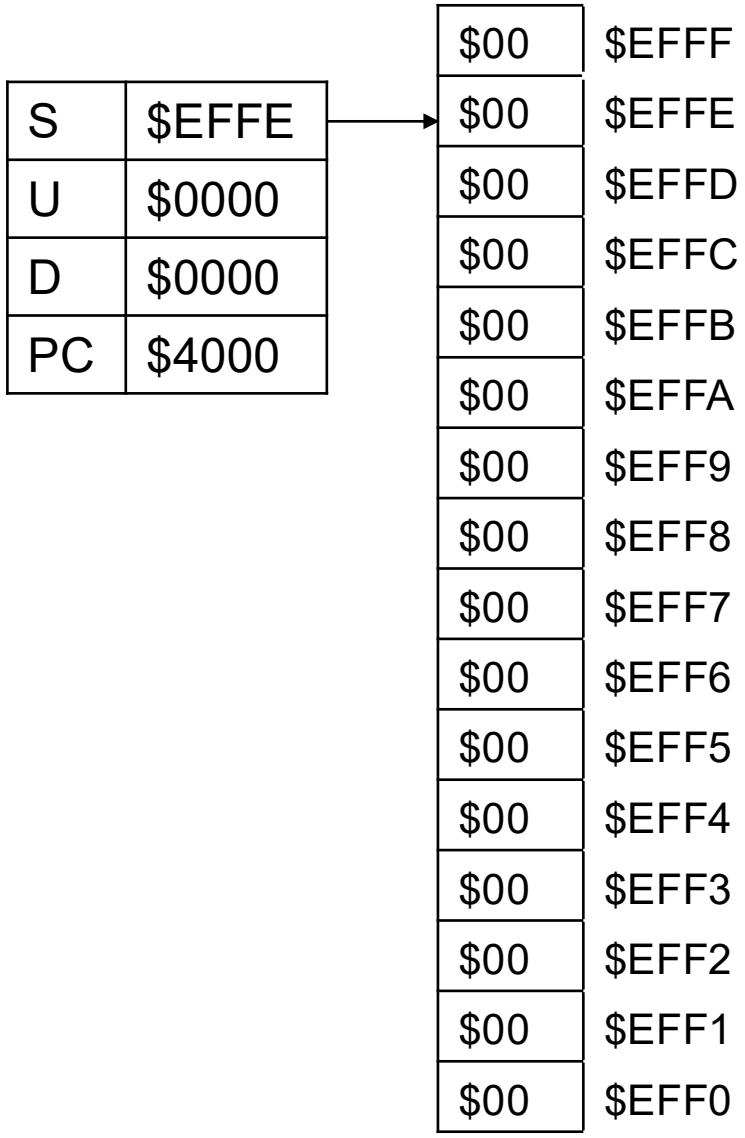
- We get the assembly:

```
* Formal parameter(s):
*       5,U:    1 byte : first: unsigned char
*       6,U:    2 bytes: second: unsigned int
LDB 5,U
CLRA
ADDD 6,U
```

- The program is accessing the parameters using ***indexed addressing mode***, relative to the top of the stack, `U`
- Return value is in `D`

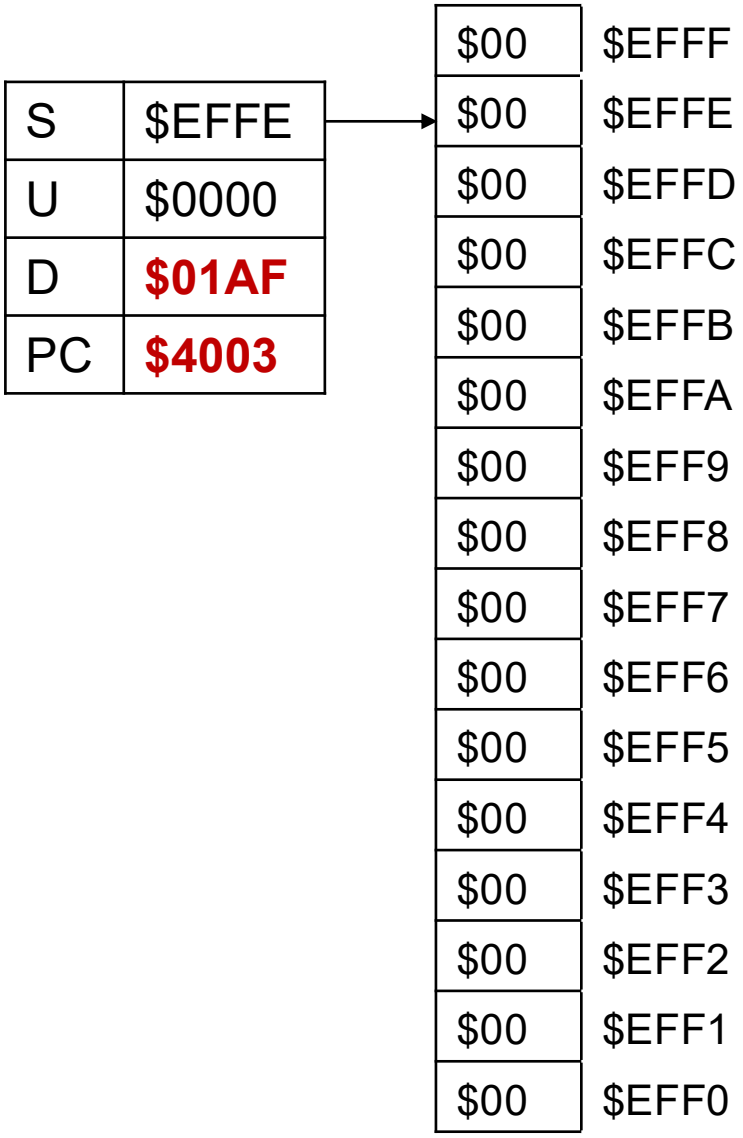
Accessing Parameters

```
4000:  CALL_ONE    LDD #$01AF
4003:              PSHS A,B
4005:              CLRA
4006:              LDB #$CC
4008:              PSHS A,B
400A:              LBSR TWO_PARAMS
400D:              LEAS $04,S
400F:              RTS
4010:  TWO_PARAM   PSHS U
4012:              LEAU ,S
4014:              LDB $05,U
4016:              CLRA
4017:              ADDD $06,U
4019:              LEAS ,U
401B:              PULS U,PC
```



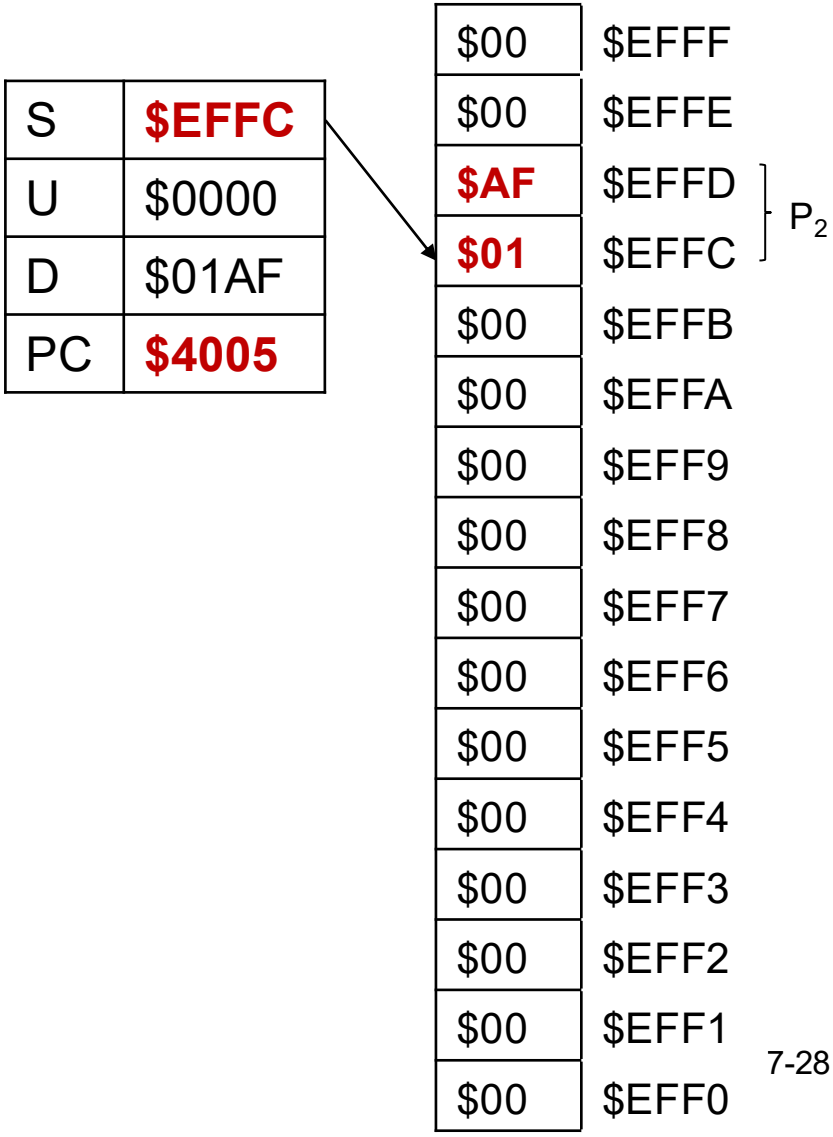
Accessing Parameters

```
4000:   CALL_ONE    LDD #$01AF
4003:               PSHS A,B
4005:               CLRA
4006:               LDB #$CC
4008:               PSHS A,B
400A:               LBSR TWO_PARAMS
400D:               LEAS $04,S
400F:               RTS
4010:   TWO_PARAM   PSHS U
4012:               LEAU ,S
4014:               LDB $05,U
4016:               CLRA
4017:               ADDD $06,U
4019:               LEAS ,U
401B:               PULS U,PC
```



Accessing Parameters

```
4000:   CALL_ONE    LDD #$01AF
4003:               PSHS A,B
4005:               CLRA
4006:               LDB #$CC
4008:               PSHS A,B
400A:               LBSR TWO_PARAMS
400D:               LEAS $04,S
400F:               RTS
4010:   TWO_PARAM   PSHS U
4012:               LEAU ,S
4014:               LDB $05,U
4016:               CLRA
4017:               ADDD $06,U
4019:               LEAS ,U
401B:               PULS U,PC
```



Accessing Parameters

```
4000:  CALL_ONE    LDD #$01AF
4003:              PSHS A,B
4005:              CLRA
4006:              LDB #$CC
4008:              PSHS A,B
400A:              LBSR TWO_PARAMS
400D:              LEAS $04,S
400F:              RTS
4010:  TWO_PARAM   PSHS U
4012:              LEAU ,S
4014:              LDB $05,U
4016:              CLRA
4017:              ADDD $06,U
4019:              LEAS ,U
401B:              PULS U,PC
```

S	$EFFC
U	$0000
D	$**00**AF
PC	**$4006**

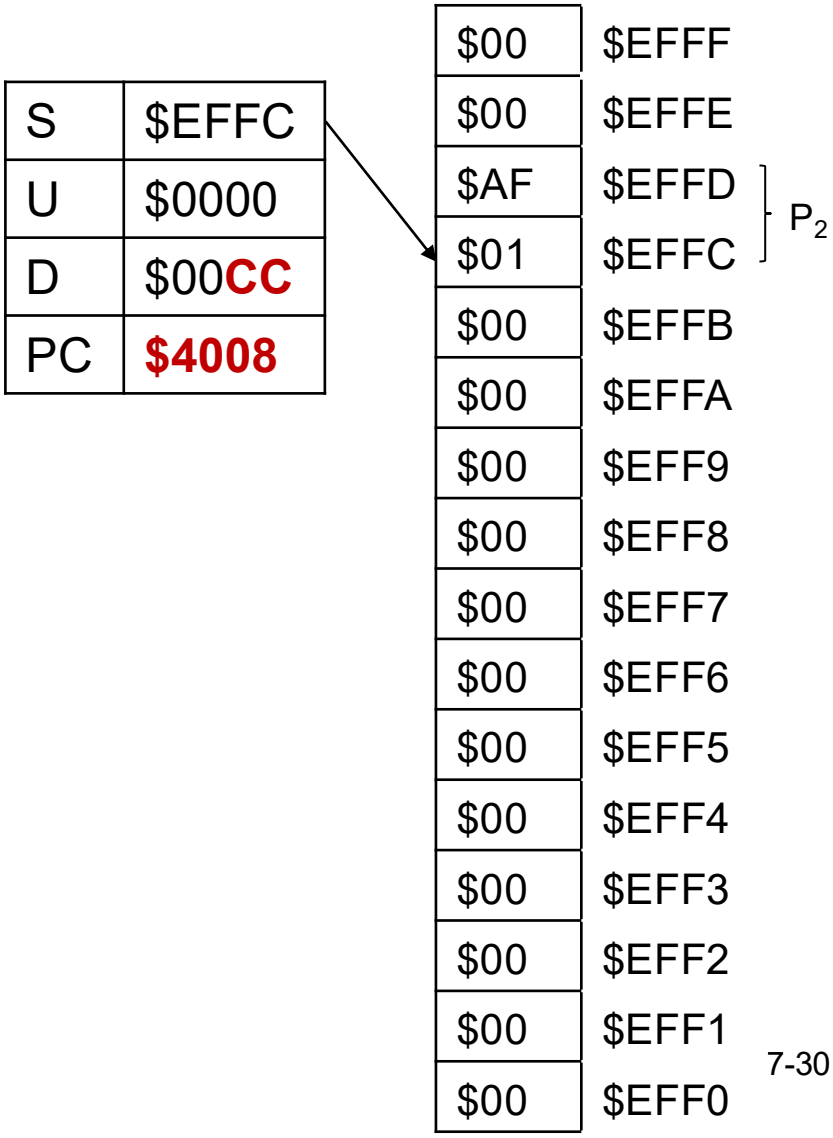
$00	$EFFF
$00	$EFFE
$AF	$EFFD
$01	$EFFC
$00	$EFFB
$00	$EFFA
$00	$EFF9
$00	$EFF8
$00	$EFF7
$00	$EFF6
$00	$EFF5
$00	$EFF4
$00	$EFF3
$00	$EFF2
$00	$EFF1
$00	$EFF0

P_2

Accessing Parameters

```
4000:   CALL_ONE    LDD #$01AF
4003:               PSHS A,B
4005:               CLRA
4006:               LDB #$CC
4008:               PSHS A,B
400A:               LBSR TWO_PARAMS
400D:               LEAS $04,S
400F:               RTS
4010:   TWO_PARAM   PSHS U
4012:               LEAU ,S
4014:               LDB $05,U
4016:               CLRA
4017:               ADDD $06,U
4019:               LEAS ,U
401B:               PULS U,PC
```

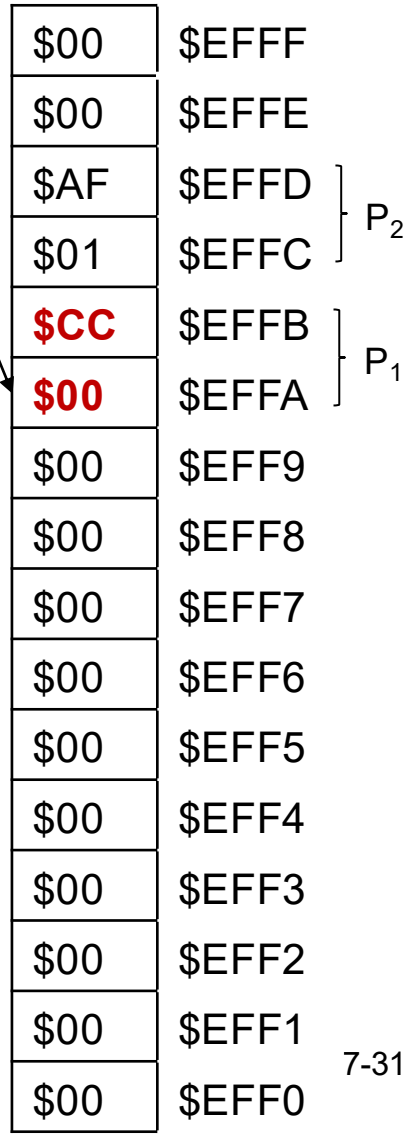
S	$EFFC
U	$0000
D	$00**CC**
PC	**$4008**



Accessing Parameters

```
4000:   CALL_ONE    LDD #$01AF
4003:               PSHS A,B
4005:               CLRA
4006:               LDB #$CC
4008:               PSHS A,B
400A:               LBSR TWO_PARAMS
400D:               LEAS $04,S
400F:               RTS
4010:   TWO_PARAM   PSHS U
4012:               LEAU ,S
4014:               LDB $05,U
4016:               CLRA
4017:               ADDD $06,U
4019:               LEAS ,U
401B:               PULS U,PC
```

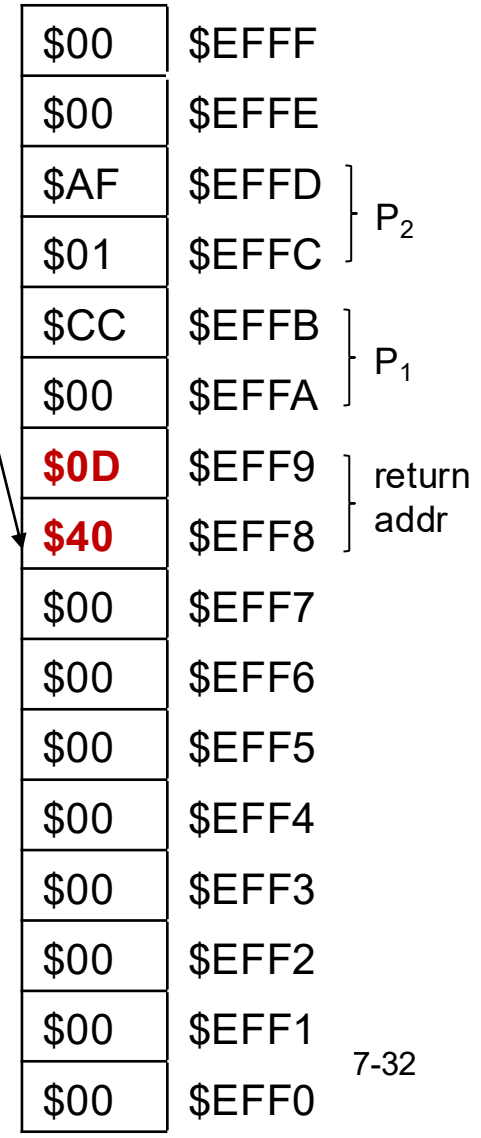
S	**$EFFA**
U	$0000
D	$00CC
PC	**$400A**



Accessing Parameters

```
4000:   CALL_ONE    LDD #$01AF
4003:               PSHS A,B
4005:               CLRA
4006:               LDB #$CC
4008:               PSHS A,B
400A:               LBSR TWO_PARAMS
400D:               LEAS $04,S
400F:               RTS
4010:   TWO_PARAM   PSHS U
4012:               LEAU ,S
4014:               LDB $05,U
4016:               CLRA
4017:               ADDD $06,U
4019:               LEAS ,U
401B:               PULS U,PC
```

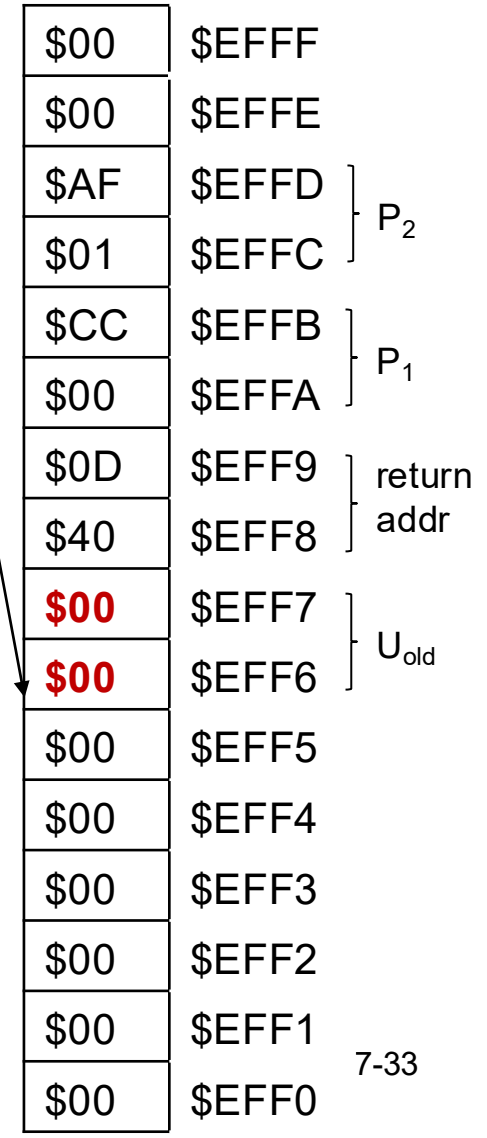
S	**$EFF8**
U	$0000
D	$00CC
PC	**$4010**



Accessing Parameters

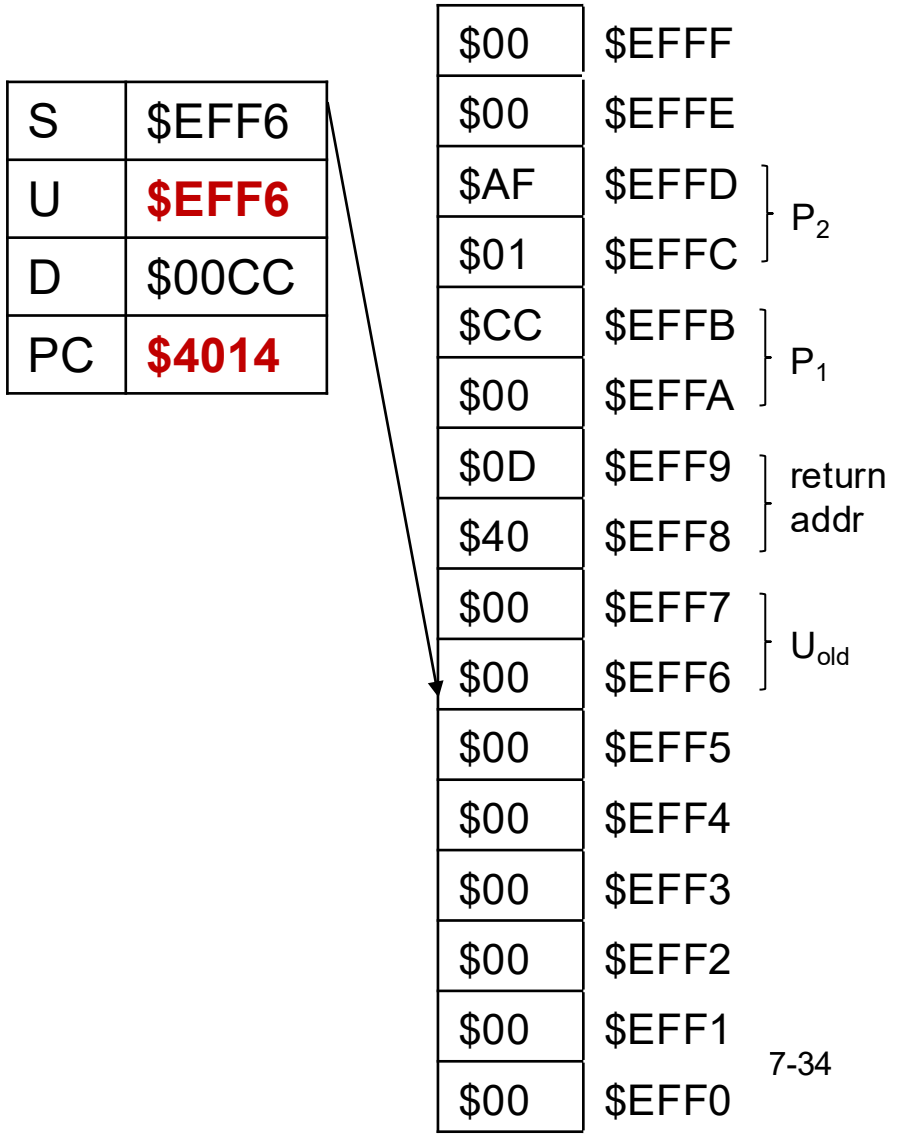
```
4000:   CALL_ONE    LDD #$01AF
4003:               PSHS A,B
4005:               CLRA
4006:               LDB #$CC
4008:               PSHS A,B
400A:               LBSR TWO_PARAMS
400D:               LEAS $04,S
400F:               RTS
4010:   TWO_PARAM   PSHS U
4012:               LEAU ,S
4014:               LDB $05,U
4016:               CLRA
4017:               ADDD $06,U
4019:               LEAS ,U
401B:               PULS U,PC
```

S	**$EFF6**
U	$0000
D	$00CC
PC	**$4012**



Accessing Parameters

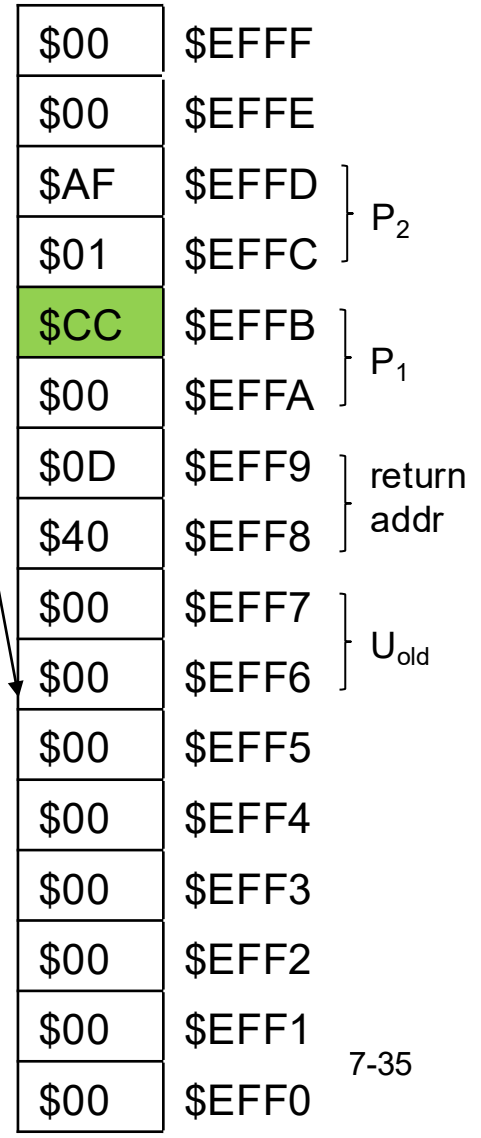
```
4000:   CALL_ONE    LDD #$01AF
4003:               PSHS A,B
4005:               CLRA
4006:               LDB #$CC
4008:               PSHS A,B
400A:               LBSR TWO_PARAMS
400D:               LEAS $04,S
400F:               RTS
4010:   TWO_PARAM   PSHS U
4012:               LEAU ,S
4014:               LDB $05,U
4016:               CLRA
4017:               ADDD $06,U
4019:               LEAS ,U
401B:               PULS U,PC
```



Accessing Parameters

```
4000:   CALL_ONE    LDD #$01AF
4003:               PSHS A,B
4005:               CLRA
4006:               LDB #$CC
4008:               PSHS A,B
400A:               LBSR TWO_PARAMS
400D:               LEAS $04,S
400F:               RTS
4010:   TWO_PARAM   PSHS U
4012:               LEAU ,S
4014:               LDB $05,U
4016:               CLRA
4017:               ADDD $06,U
4019:               LEAS ,U
401B:               PULS U,PC
```

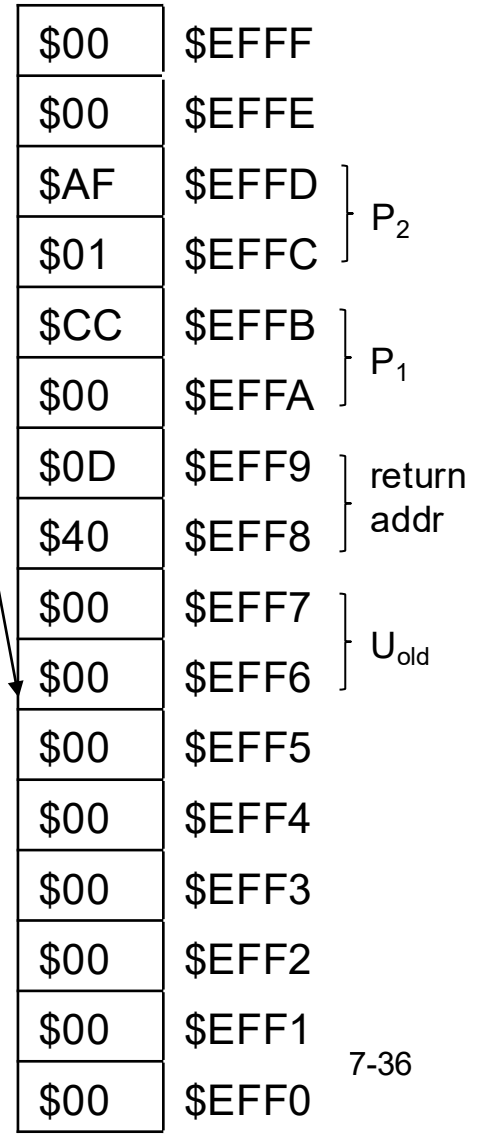
S	$EFF6
U	$EFF6
D	$00**CC**
PC	**$4016**



Accessing Parameters

```
4000:  CALL_ONE    LDD #$01AF
4003:              PSHS A,B
4005:              CLRA
4006:              LDB #$CC
4008:              PSHS A,B
400A:              LBSR TWO_PARAMS
400D:              LEAS $04,S
400F:              RTS
4010:  TWO_PARAM   PSHS U
4012:              LEAU ,S
4014:              LDB $05,U
4016:              CLRA
4017:              ADDD $06,U
4019:              LEAS ,U
401B:              PULS U,PC
```

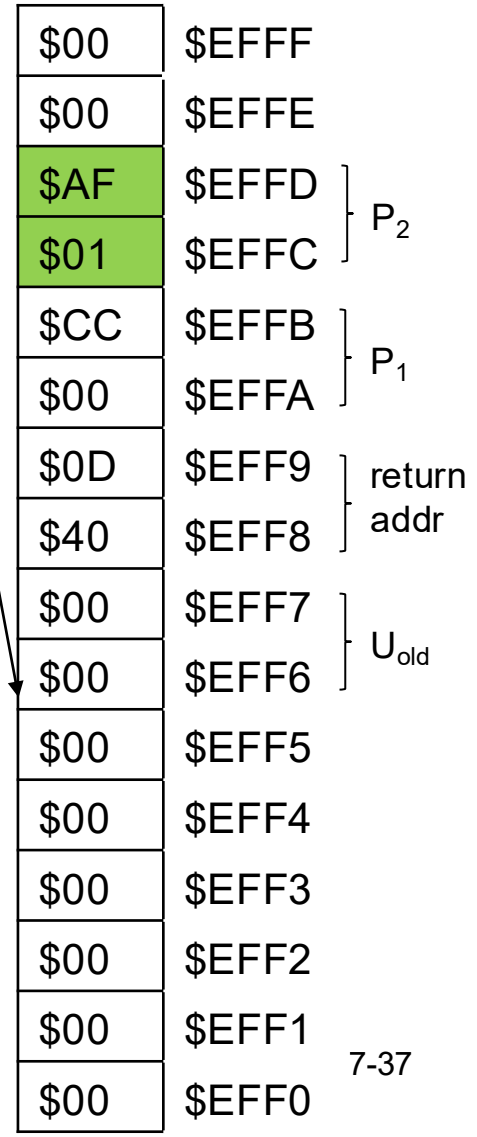
S	$EFF6
U	$EFF6
D	**$00**CC
PC	**$4017**



Accessing Parameters

```
4000:   CALL_ONE    LDD #$01AF
4003:               PSHS A,B
4005:               CLRA
4006:               LDB #$CC
4008:               PSHS A,B
400A:               LBSR TWO_PARAMS
400D:               LEAS $04,S
400F:               RTS
4010:   TWO_PARAM   PSHS U
4012:               LEAU ,S
4014:               LDB $05,U
4016:               CLRA
4017:               ADDD $06,U
4019:               LEAS ,U
401B:               PULS U,PC
```

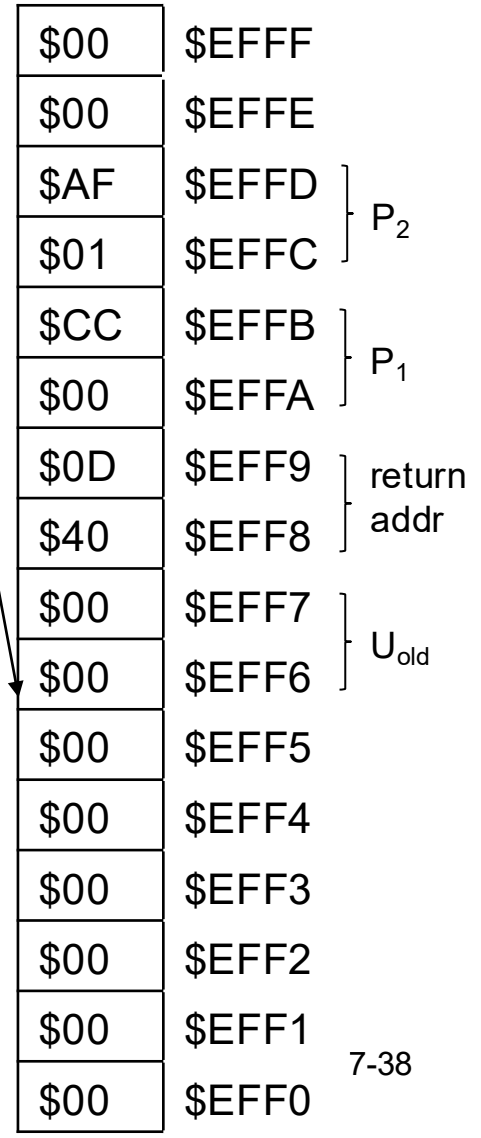
S	$EFF6
U	$EFF6
D	**$027B**
PC	**$4019**



Accessing Parameters

```
4000:   CALL_ONE    LDD #$01AF
4003:               PSHS A,B
4005:               CLRA
4006:               LDB #$CC
4008:               PSHS A,B
400A:               LBSR TWO_PARAMS
400D:               LEAS $04,S
400F:               RTS
4010:   TWO_PARAM   PSHS U
4012:               LEAU ,S
4014:               LDB $05,U
4016:               CLRA
4017:               ADDD $06,U
4019:               LEAS ,U
401B:               PULS U,PC
```

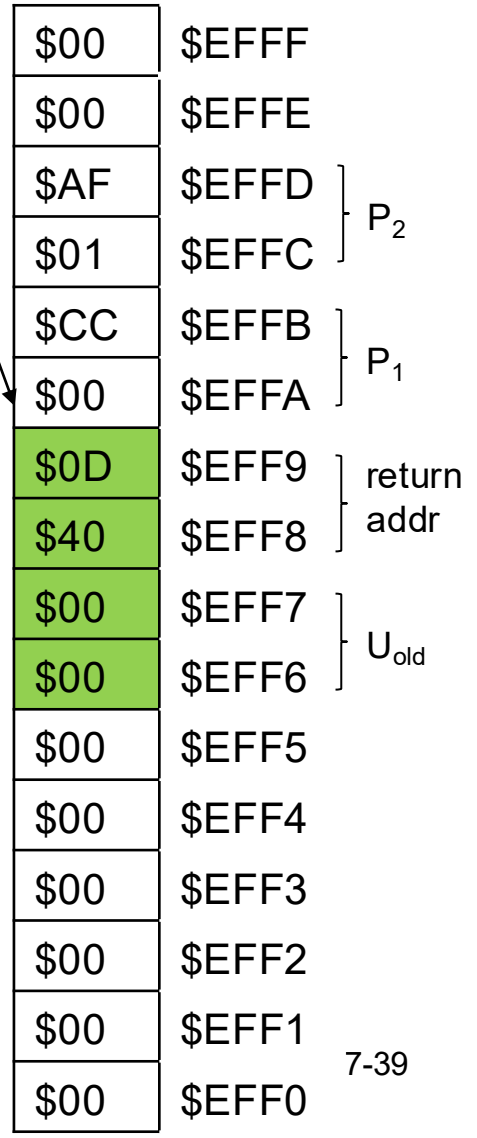
S	**$EFF6**
U	$EFF6
D	$027B
PC	**$401B**



Accessing Parameters

```
4000:   CALL_ONE    LDD #$01AF
4003:               PSHS A,B
4005:               CLRA
4006:               LDB #$CC
4008:               PSHS A,B
400A:               LBSR TWO_PARAMS
400D:               LEAS $04,S
400F:               RTS
4010:   TWO_PARAM   PSHS U
4012:               LEAU ,S
4014:               LDB $05,U
4016:               CLRA
4017:               ADDD $06,U
4019:               LEAS ,U
401B:               PULS U,PC
```

S	**$EFFA**
U	**$0000**
D	$027B
PC	**$400D**



Accessing Parameters

```
4000:  CALL_ONE    LDD #$01AF
4003:              PSHS A,B
4005:              CLRA
4006:              LDB #$CC
4008:              PSHS A,B
400A:              LBSR TWO_PARAMS
400D:              LEAS $04,S
400F:              RTS
4010:  TWO_PARAM   PSHS U
4012:              LEAU ,S
4014:              LDB $05,U
4016:              CLRA
4017:              ADDD $06,U
4019:              LEAS ,U
401B:              PULS U,PC
```

S	**$EFFE**
U	$0000
D	$027B
PC	**$400F**

$00	$EFFF
$00	$EFFE
$AF	$EFFD
$01	$EFFC
$CC	$EFFB
$00	$EFFA
$0D	$EFF9
$40	$EFF8
$00	$EFF7
$00	$EFF6
$00	$EFF5
$00	$EFF4
$00	$EFF3
$00	$EFF2
$00	$EFF1
$00	$EFF0

Accessing Parameters

```
4000:  CALL_ONE    LDD #$01AF
4003:              PSHS A,B
4005:              CLRA
4006:              LDB #$CC
4008:              PSHS A,B
400A:              LBSR TWO_PARAMS
400D:              LEAS $04,S
400F:              RTS
4010:  TWO_PARAM   PSHS U
4012:              LEAU ,S
4014:              LDB $05,U
4016:              CLRA
4017:              ADDD $06,U
4019:              LEAS ,U
401B:              PULS U,PC
```

S	**$F000**
U	$0000
D	$027B
PC	**$0000**

$00	$EFFF
$00	$EFFE
$AF	$EFFD
$01	$EFFC
$CC	$EFFB
$00	$EFFA
$0D	$EFF9
$40	$EFF8
$00	$EFF7
$00	$EFF6
$00	$EFF5
$00	$EFF4
$00	$EFF3
$00	$EFF2
$00	$EFF1
$00	$EFF0

Local Variables

- Local variables are stored on the stack too, and indexed addressing is used to access them
- The pre-amble reserves space on the stack and the post-amble cleans it up afterwards

```
uint16_t one_param(uint16_t xyzzy)
{
uint16_t val = xyzzy;
return val;
}
```

```
_one_param
    PSHS U
    LEAU ,S
    LEAS -2,S
    * Formal parameter(s):
    *      4,U:    2 bytes: xyzzy
    * Local non-static variable(s):
    *     -2,U:    2 bytes: val
    LDD 4,U
    STD -2,U
    LEAS ,U
    PULS U,PC
```

Homework

- The CMOC manual has a section discussing the calling convention. See: http://gvlsywt.cluster051.hosting.ovh.net/dev/cmoc-manual.html
- How does CMOC pass a `struct` by value?
- Can you return a `struct` from a routine? If so, then how does CMOC do this?